cotomi Act: Learning to Automate Work by Watching You

Masafumi Oyamada Kunihiro Takeoka Kosuke Akimoto Ryoma Obara
Masafumi Enomoto Haochen Zhang Daichi Haraguchi Takuya Tamura

NEC Corporation
{oyamada, k_takeoka, kosuke_a, ryoma-obara,
masafumi-enomoto, haochen-zhang, daichi-haraguchi, tamura-takuya}@nec.com

Abstract

What if a browser agent could learn your work simply by watching you do it? We present cotomi Act, a browser-based computer-using agent that combines reliable multi-step task execution with persistent organizational knowledge learned from user behavior. For execution, an agent scaffold with adaptive lazy observation, verbal-diff-based history compression, coarse-grained actions, and test-time scaling via best-of-N action selection achieves 80.4% on the 179-task WebArena human-evaluation subset, exceeding the reported 78.2% human baseline. For organizational knowledge, a behavior-to-knowledge pipeline passively observes the user's browsing and progressively abstracts it into artifacts (task boards, wiki) exposed through a shared workspace editable by both user and agent. A controlled proxy evaluation confirms that task success improves as behavior-derived knowledge accumulates. In our live demonstration, attendees interact with the system in a real browser, issuing tasks and observing end-to-end autonomous execution and shared knowledge management.

CCS Concepts

• **Computing methodologies** → **Intelligent agents**; • **Human-centered computing** → *Web-based interaction.*

Keywords

web agent, browser automation, computer-using agent, shared knowledge workspace, human-agent collaboration

ACM Reference Format:
Masafumi Oyamada, Kunihiro Takeoka, Kosuke Akimoto, Ryoma Obara, Masafumi Enomoto, Haochen Zhang, Daichi Haraguchi, and Takuya Tamura. 2026. cotomi Act: Learning to Automate Work by Watching You. In *ACM Conference on AI and Agentic Systems (CAIS '26), May 26–29, 2026, San Jose, CA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3786335.3813203

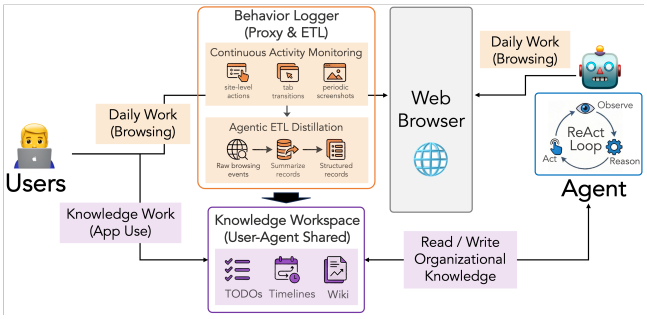

Figure 1: Unlike conventional computer-using agents that execute tasks with no knowledge of the user's organization, cotomi Act continuously learns from ordinary browsing. The Behavior Logger (top) monitors user activity and distills it via an agentic ETL pipeline into structured artifacts—tasks, timelines, and wiki pages—stored in a shared Knowledge Workspace (bottom) that both the user and the agent can read and edit. The Agent (right) consults these artifacts during its ReAct loop, acting as a situated co-worker rather than a stateless executor.

1 Introduction

Large language models (LLMs) have grown dramatically more capable at reasoning, planning, and tool use, giving rise to *computer-using agents* (CUAs) that can operate web browsers on behalf of users [3, 7, 14]. Yet strong reasoning alone is insufficient for a reliable co-worker. Every organization operates with tacit knowledge—who approves what, what internal jargon means, and which tasks are already in progress—that is rarely written down but essential for getting things done. Today's computer-using agents ignore this reality. To function as a genuine co-worker, a CUA needs two ingredients that today's systems still struggle to combine: **(1) strong reasoning ability** for reliably carrying out complex, multi-step web tasks, and **(2) organizational knowledge**—the tacit preferences, unwritten workflows, and conventions that exist in every organization but are rarely documented.

Prior systems fall short on both counts. On the **reasoning** side, even well-specified benchmark tasks remain difficult: the best reported agent reaches 71.6% on WebArena [8], still below the 78.2% human baseline [30]. Long-horizon tasks in complex web UIs demand repeated observation and action, producing lengthy context that strains model reasoning. On the **organizational-knowledge** side, current CUAs execute each task knowing nothing about local workflows, preferences, or pending work. Without such context, an agent faces a costly dilemma on every ambiguous instruction: ask frequent clarification questions at the expense of efficiency, or silently guess and risk irreversible errors. Moreover, organizational

CAIS '26, San Jose, CA, USA

ACM ISBN 979-8-4007-2415-2/2026/05
https://doi.org/10.1145/3786335.3813203

knowledge spans a wide spectrum—from relatively stable conventions and approval rules to rapidly changing states such as who is assigned to which task right now. A natural response is to let the agent retrieve internal documents via RAG-like mechanisms [23], but enterprise repositories are vast and often stale, and much of the knowledge that matters most—how people actually work—is never written down at all.

We present cotomi Act, a browser-based CUA that addresses both requirements. For **strong reasoning ability**, the system employs a carefully engineered agent scaffold (Section 2) combining adaptive lazy observation, verbal-diff-based history compression, coarse-grained actions, and test-time scaling via best-of-N action selection and task decomposition [11, 12]—together keeping the context window focused and enabling reliable execution over long-horizon web tasks. On WebArena [30], this scaffold achieves 80.4% on the 179-task human-evaluation subset [13], exceeding the reported human baseline of 78.2% [30] (Table 1). For **tacit / organizational knowledge**, cotomi Act takes a different approach: it *learns from user behavior*. A background activity monitor observes the user's ordinary browsing, and an LLM-based pipeline (Section 2) progressively distills those observations—which documents the user consults, in what context, and what tasks they are working on—into a working model of how the user works. This lets the agent navigate the vast sea of enterprise information along paths the user has already carved. The resulting knowledge is surfaced as shared artifacts such as task boards and wiki pages that both the user and the agent can read and edit; through routine use, the user naturally adds, corrects, and removes entries, so that even undocumented knowledge finds its way into the agent's grounding.

Our contributions are:

(1) cotomi Act, a browser-based computer-using agent that achieves 80.4% on the 179-task WebArena human-evaluation subset—exceeding the reported human baseline (78.2%)—through an agent scaffold combining adaptive lazy observation, verbal-diff-based history compression, coarse-grained actions, and test-time scaling via best-of-N action selection over multi-step web tasks.
(2) A behavior-to-knowledge pipeline that passively observes a user's ordinary browsing, progressively abstracts raw events into reusable knowledge artifacts (task boards, wiki, activity timelines), and exposes them through a shared workspace where both the user and the agent can read and edit—enabling persistent learning and transparent human–agent alignment.
(3) A controlled proxy evaluation showing that downstream task success generally improves as the agent accumulates more behavior-derived knowledge—empirically validating the behavior-to-knowledge pipeline.

The remainder of this paper describes the system architecture (Section 2), evaluation (Section 3), and live demonstration scenario (Section 4).

2 System Architecture

Figure 1 shows the architecture of cotomi Act. The design mirrors the two requirements identified in Section 1: an *agent scaffold* that provides strong reasoning over multi-step web tasks, and a *behavior-to-knowledge pipeline* that continuously converts observed user activity into organizational knowledge the agent can act on. A *shared knowledge workspace* bridges the two, storing artifacts that both the user and the agent read and edit. All three are delivered through a single browser extension, with the agent scaffold and behavior logger running as backend services described below.

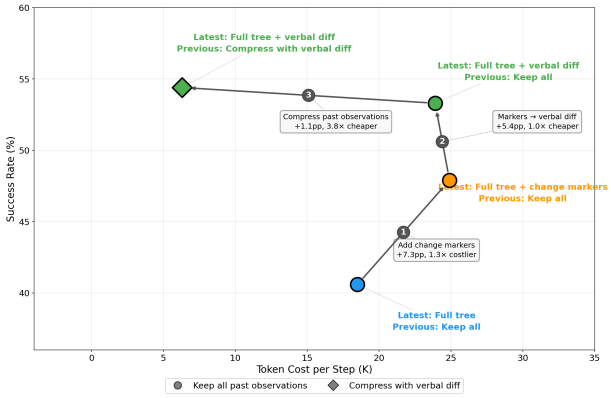

Figure 2: Observation and history design trajectory on WebArena-Verified (Gemma-4-31B-IT, 82 tasks, 3 runs). Each arrow traces one design change in accuracy–token-cost space. Applying verbal diffs to the current observation improves accuracy, while using them to replace stale history reduces cost, jointly moving the scaffold toward the upper-left (higher accuracy, lower cost).

2.1 Agent Scaffold

The web automation agent follows a ReAct-style loop [26] of observation, reasoning, and action, orchestrated by what we call the *agent scaffold*—the infrastructure that manages browser state, composes prompts, and routes actions. At each step, the model's context window carries two kinds of information: the **current observation** (what the agent sees right now) and the **execution history** (what happened in prior steps). Both compete for the same finite token budget, and their design interacts with the **action space** that determines how many steps a task requires. The central challenge is *context bloat*: feeding the model a full accessibility tree at every step, or retaining all past observations verbatim, fills the context window with stale information and degrades decision quality in later steps.

We address this through four scaffold-level mechanisms: adaptive observation, compact execution history, coarse-grained actions, and task decomposition. We validate the observation and history choices through a systematic ablation on WebArena-Verified [5] (Gemma-4-31B-IT, 39 configurations, 82 tasks × 3 runs; Figure 2). We then describe how the scaffold retrieves organizational context from the shared workspace.

Current observation. A richer current observation generally improves accuracy. Starting from a baseline of the full accessibility tree with no change information (40.6% SR), adding rule-generated *verbal diffs*—natural-language descriptions of DOM-level changes (e.g., "New element: [42] button 'Submit' ")—yields the largest single improvement (+12.7 pp to 53.3%), substantially outperforming

positional change markers (+7.3 pp). However, richer observations increase per-step token cost and latency. When latency matters more than peak accuracy, a **lazy observation** strategy offers a compelling trade-off: sending only the viewport-visible elements—and letting the agent request the full tree on demand—achieves comparable accuracy while reducing end-to-end response time from 471 s to 184 s per task (roughly 2.6× faster). cotomi Act adopts this lazy strategy by default and enriches on demand for visually complex pages.

Execution history. Retaining full past observations in the conversation is counterproductive: input tokens grow linearly with steps and reach ~57K by step 20, crowding the context window. The key insight is that the agent does not need the raw page state at step 5—it needs to know *what changed* between steps 4 and 5. Replacing stale observations with the same verbal diffs used in the current observation achieves comparable accuracy (+1.1 pp) while reducing per-step token cost by 3.8× (23.9K → 6.3K). While diff-based history representations have been shown to be token-efficient [6], we find that these natural-language change descriptions outperform standard diff formats by preserving semantic context rather than positional deltas. Notably, the current-observation and history axes partially substitute: both encode "what changed" through different channels, so their combined gain (+13.8 pp) is less than the sum of individual effects (+22.1 pp).

Coarse-grained actions. If the action space is too fine-grained, the agent needs many steps to accomplish a single intent; the resulting long context dilutes relevant information and lowers task success. If actions are too coarse, the agent loses the flexibility to handle unexpected page layouts. We resolve this by defining actions at the highest abstraction that still covers common web interactions. Scrolling illustrates the point: replacing pixel-level `scrollBy(x, y)`—which can require tens of repetitions to reach a target—with element-level `scrollInto(element)` reduces scroll sequences to a single step without sacrificing precision.

Task decomposition. Long-horizon tasks risk *context overflow*: as steps accumulate, earlier observations crowd the reasoning window and degrade decision quality. To counteract this, the scaffold decomposes a task when the reasoning model emits a structured plan indicating multiple independent sub-goals [12], spawning parallel sub-agents that each maintain a focused, compact context (see Appendix A for details).

Organizational context. Beyond controlling what the model sees during execution, the scaffold also controls what the agent knows *before* it acts. When organizational context is needed, the reasoning model explicitly invokes a `search_workspace` tool that accepts a natural-language query and returns matching task items, wiki pages, or activity-timeline entries ranked by relevance. The model decides autonomously whether and when to invoke this tool—typically when the user instruction is under-specified, references organizational jargon, or asks the agent to resume pending work. This on-demand design avoids injecting irrelevant context on every turn and bridges the gap between a generic executor and a situated co-worker. The next subsection describes how these workspace artifacts are populated from observed user behavior.

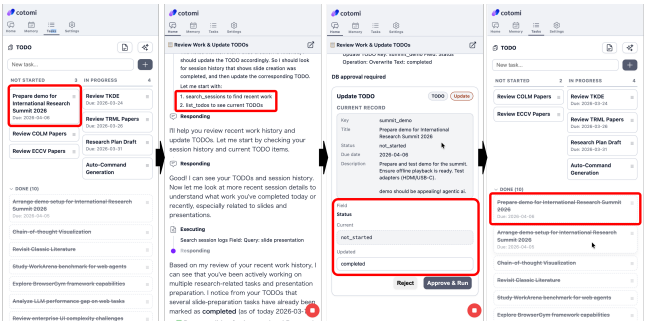

Figure 3: Bidirectional workspace curation in action. The agent reviews the user's recent browsing sessions, proposes a task status update (not_started → completed), and the user approves the change—keeping the shared workspace aligned with reality.

2.2 User Behavior Logger

Learning how the user works requires observing their actual behavior, but raw browsing logs are noisy, voluminous, and far too low-level to be useful as organizational knowledge. The behavior logger addresses this through a two-phase design: lightweight capture followed by progressive LLM-based abstraction.

In the capture phase, the activity monitor records site-level actions, tab transitions, and periodic screenshots in the background, with minimal interference to the user's workflow. A *diff-based compression* scheme deduplicates consecutive observations on the same site, reducing both storage volume and downstream noise.

In the abstraction phase, an LLM-based ETL pipeline periodically processes compressed events through three stages:

(1) Segment raw events into task-level episodes using inactivity timeouts.
(2) Summarize each segment into a structured record (screen state, actions, resulting changes) and classify it by activity category.
(3) Aggregate summaries into knowledge artifacts—task items, wiki pages, and activity timelines.

We compared a fixed-rule pipeline against an *agentic ETL* design in which the LLM autonomously decides segmentation boundaries, abstraction level, and artifact routing. In a blind comparison by two annotators across 50 browsing sessions, the agentic variant produced more accurate and coherent artifacts, an important advantage given the volume and heterogeneity of real browsing logs.

Privacy. The behavior logger operates on a strictly *opt-in* basis: recording is disabled by default and activates only after explicit user consent, and the user can pause or stop recording at any time. All captured data is processed through a PII-masking stage that redacts personally identifiable information before artifacts are written to the shared workspace.

2.3 Shared Knowledge Workspace

The central design principle of the workspace is that every knowledge artifact is a *first-class user application*—simultaneously human-readable and agent-queryable—rather than an agent-internal memory [15, 20, 21]. This makes bidirectional editing natural: the agent

Table 1: Task success rate (%) on WebArena. Human performance was measured on a 179-task subset of the 812-task benchmark [30]; we evaluate on the same subset (*Human subset*) to enable direct comparison, and additionally report full-benchmark scores (*All tasks*). †Recomputed by us from publicly available results. "site" = site-specific prompts covering UI elements and navigation; "fmt" = answer-format clarifications only.

System	Hints	All tasks	Human subset
SteP [18]	site	33.5	40.8†
OpenAI Operator [14]	site	58.1	—
CUGA [17]	site	61.7	64.3†
OpAgent [8]	site	71.6	74.9†
Human [30]	—	—	78.2
cotomi Act (base)	fmt	74.4	76.5
cotomi Act (+TTS)	fmt	75.7	**80.4**

bootstraps artifacts from observed behavior, and the user inspects, edits, and approves them through familiar interfaces. The workspace currently provides three artifact types:

- **Activity timeline**: a calendar-based display annotated with activity tags (e.g., "communication," "research," "reporting") and durations; users can drill down into any tag to review detailed descriptions.
- **Task board**: a shared task database that both agent and user can read and write—the agent proposes updates by reviewing recent activity, while the user retains approval authority over every change.
- **Wiki**: a collection of pages capturing organizational rules, conventions, and procedural notes (e.g., approval workflows, naming conventions, preferred vendors) that the agent distills from observed behavior and the user curates over time.

This bidirectional loop (Figure 3) is essential to organizational understanding: the agent's inferences from observation alone are inevitably incomplete, so exposing artifacts for human curation keeps the agent's knowledge aligned with the user's evolving reality.

3 Evaluation

We evaluate cotomi Act against the two ingredients introduced in Section 1: reliable task execution (Section 3.1) and behavioral knowledge (Section 3.2).

3.1 Task Execution

We evaluate the execution engine on WebArena [30], a benchmark of 812 realistic web tasks spanning five domains (e-commerce, forums, content management, maps, and GitLab).

Setup. WebArena tasks are well-specified single-session instructions that isolate execution capability from organizational context, making it a suitable testbed for the execution requirement. To ensure a fair comparison with the human baseline, we evaluate on the same 179-task subset used for the human performance study [30].[1] Because we found that the WebArena automatic scorer produces false positives and false negatives on a non-trivial fraction of tasks, three annotators manually verified every automatic judgment [27]. Table 1 compares cotomi Act against representative systems. Notably, cotomi Act (marked "fmt") uses only answer-format clarifications, without any site-specific UI or navigation hints used by other systems.

Results. cotomi Act achieves a success rate of 80.4% on this 179-task subset (75.7% on the full benchmark), exceeding the reported human baseline of 78.2% [30]. The base scaffold—adaptive lazy observation, verbal-diff-based history compression, and coarse-grained actions—already reaches 76.5%. Adding test-time scaling (+TTS), i.e. best-of-N action selection and task decomposition [11, 12], yields a further 3.9 percentage-point gain, confirming that reducing per-step variance through pre-execution consensus is critical for long-horizon web tasks (Appendix A provides additional details).

3.2 Behavioral Knowledge

We evaluate whether accumulated behavioral knowledge improves task success, and which format of knowledge is most effective, using a controlled study on WorkArena-L1 [4].

Setup. WorkArena-L1 tasks resemble repetitive knowledge-work procedures in enterprise systems (filtering records, filling forms, ordering catalog items), making them sensitive to procedural familiarity rather than exploratory reasoning. We collect successful agent trajectories across WorkArena-L1's six task categories as a proxy for observed user behavior. This is a controlled approximation: real user traces would additionally contain failures, interruptions, and exploratory detours, so the pipeline's robustness to such noise remains to be validated in a full deployment study. We vary *domain coverage*—the fraction of categories whose behavioral knowledge is available to the agent—from 0% (no knowledge) to 100% (all six categories, ∼277 hints total); see Appendix B for full protocol details. The three formats—*trajectory* (full trace), *script* (procedural steps), and *insight* (abstract summary)—represent progressive abstraction levels produced by the agentic ETL pipeline (Section 2); all are stored as wiki pages and retrieved on demand via the scaffold's workspace-search tool.

Results. Figure 4 shows that task success generally improves with coverage for all three formats, gaining up to +10 percentage points over the zero-coverage baseline of ≈51% (each point averaged over six random category orderings; see Appendix B for protocol details)—confirming that behavioral knowledge is broadly beneficial. The most effective format, however, depends on how much user behavior is available: raw trajectories actually *degrade* performance at low coverage (−2 pp at 17% coverage) before recovering to become the strongest format at high coverage, whereas scripts and insights remain stable across the entire range.

Implications for system design. Raw trajectories achieve the highest ceiling but are impractical at scale; the abstraction–detail trade-off motivates cotomi Act's agentic ETL pipeline, which distills raw traces into compact, retrievable artifacts.

[1] Human trajectories: https://github.com/web-arena-x/webarena/blob/main/resources/README.md

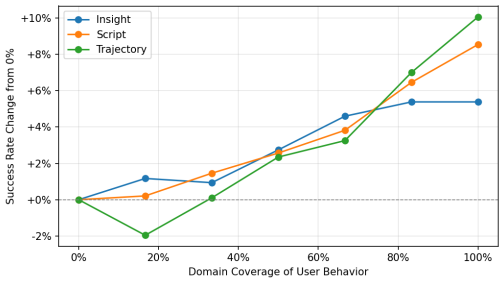

Figure 4: Change in success rate relative to the zero-coverage baseline (i.e., the agent uses no behavioral knowledge) as domain coverage of user behavior increases. More coverage improves downstream task success, while the most effective format depends on how much user behavior is available.

A concrete episode illustrates the full observe–extract–act chain. In the service-catalog domain, the behavior logger records a user completing catalog-ordering tasks (navigating to *Service Catalog* → *Hardware*, selecting a product, filling configuration fields, and clicking *Order Now*). The agentic ETL pipeline segments these sessions and distills each into a procedural script; 48 such scripts are generated across all catalog-ordering templates. When the agent subsequently encounters a catalog-ordering task, it invokes `search_workspace`, retrieves the matching script, and follows its steps. Before receiving any catalog-ordering scripts the agent succeeds on ≈75% of these tasks; after the 48 scripts become available, success jumps to ≈99% (averaged across six random category orderings). This sharp improvement confirms that the behavioral-knowledge pipeline produces artifacts with direct causal impact on task execution. Because these artifacts live in the shared knowledge workspace, the user can inspect and correct them, closing the gap that pure abstraction inevitably opens.

4 Demonstration Scenario

The live demonstration allows attendees to interact with cotomi Act through a browser equipped with the extension, connected to a hosted agent backend. Three scenarios showcase the system's core capabilities in approximately 10 minutes.

Scenario 1: Ad-hoc task automation. Attendees type a natural-language instruction (e.g., "Search for recent AI conferences in 2026 and list the submission deadlines") and observe the agent autonomously navigating, extracting information, and recovering from errors in real time.

Scenario 2: Inspecting the shared knowledge workspace. Using a pre-populated activity history, the demonstrator shows the calendar-based memory view with time-use analytics, drills down into a specific activity tag, and inspects generated artifacts such as task items and wiki entries.

Scenario 3: Workspace-guided execution. The demonstrator edits a task item to refine its priority or add contextual notes, then instructs the agent: "Look at my task list, pick the highest-priority task you can help with, and work on it." The agent consults the task board, presents candidate tasks via a multiple-choice interaction, and upon user selection begins executing the chosen task—demonstrating the bidirectional editing loop and full observe–extract–act cycle. This scenario highlights the system's core novelty: persistent, user-editable organizational knowledge that directly shapes the agent's next action.

5 Related Work

Web Agent Systems. WebArena [30] and OSWorld [22] established realistic benchmarks, revealing a large gap between LLM-based agents and human operators. Despite advances in structured reasoning [1, 25] and vision-based approaches [9, 29], a recent meta-evaluation [24] found most agents plateau well below human performance; OpAgent [8] raised the ceiling to 71.6% but still falls short. Commercial CUAs [3, 7, 14] demonstrated general-purpose browser automation but do not expose persistent, user-editable organizational context.

Agent Memory Architectures. Several works maintain persistent memory for LLM agents: hierarchical memory management [15], code-level skill libraries [20], verbal self-reflections [16, 28], and reusable workflow routines [21]. These systems keep memory inside the agent, capturing the agent's own experience but not the user's actual work patterns. cotomi Act instead learns from observed browser behavior and externalizes behavioral knowledge as typed, user-editable artifacts; routine user maintenance keeps the agent aligned without explicit synchronization.

Human-AI Collaboration. The shared knowledge workspace draws on principles from human-AI collaboration. Horvitz [10] established foundations for mixed-initiative interaction. Andrews et al. [2] showed that shared mental models underpin effective human-AI teaming. Star and Griesemer [19] introduced *boundary objects*—artifacts shared across different communities yet interpretable by each on its own terms. cotomi Act's shared artifacts serve as boundary objects: the human sees a task management tool, while the agent sees an action source grounding its next execution. This design achieves transparency by construction—the agent's knowledge is externalized as everyday artifacts rather than requiring post-hoc explanation—and supports the collaborative refinement loop.

6 Conclusion

cotomi Act demonstrates that a browser-based AI co-worker requires two complementary capabilities: reliable multi-step web execution, and organizational knowledge acquired from everyday user behavior. The agent scaffold achieves 80.4% on the WebArena human-evaluation subset—exceeding the reported human baseline—while a progressive abstraction pipeline converts raw browsing activity into behavioral knowledge that both human and agent maintain as shared artifacts through routine use. A controlled proxy study confirms that task success improves as this behavior-derived knowledge accumulates, validating learning from observation as a practical path to organizational grounding.

References

[1] Saaket Agashe, Jiuzhou Han, Shuyu Gan, Jiachen Yang, Ang Li, and Xin Eric Wang. 2025. Agent S: An Open Agentic Framework that Uses Computers Like a Human. In *International Conference on Learning Representations*. OpenReview.net, Singapore, 24 pages.

[2] Robert W. Andrews, J. Mason Lilly, Divya Srivastava, and Karen M. Feigh. 2023. The Role of Shared Mental Models in Human-AI Teams: A Theoretical Review. *Theoretical Issues in Ergonomics Science* 24, 2 (2023), 129–175.

[3] Anthropic. 2024. Introducing Computer Use, a New Claude 3.5 Sonnet, and Claude 3.5 Haiku. https://www.anthropic.com/news/3-5-models-and-computer-use.

[4] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vázquez, Nicolas Chapados, and Alexandre Lacoste. 2024. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? *arXiv preprint arXiv:2403.07718* (2024), 27 pages.

[5] Amine El hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. 2025. WebArena Verified: Reliable Evaluation for Web Agents. In *Workshop on Scaling Environments for Agents*. https://openreview.net/forum?id=94tlGxmqkN

[6] Masafumi Enomoto, Ryoma Obara, Haochen Zhang, and Masafumi Oyamada. 2026. Read More, Think More: Revisiting Observation Reduction for Web Agents. arXiv:2604.01535 [cs.CL]

[7] Google. 2024. Project Mariner. https://deepmind.google/technologies/project-mariner/.

[8] Yuyu Guo, Wenjie Yang, Siyuan Yang, Ziyang Liu, Cheng Chen, Yuan Wei, Yun Hu, Yang Huang, Guoliang Hao, Dongsheng Yuan, Jianming Wang, Xin Chen, Hang Yu, Lei Lei, and Peng Di. 2026. OpAgent: Operator Agent for Web Navigation. *arXiv preprint arXiv:2602.13559* (2026).

[9] Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2024. A Real-World WebAgent with Planning, Long Context Understanding, and Program Synthesis. In *International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 23 pages.

[10] Eric Horvitz. 1999. Principles of Mixed-Initiative User Interfaces. In *CHI '99: Conference on Human Factors in Computing Systems*. ACM, New York, NY, USA, 159–166.

[11] Junpei Komiyama, Daisuke Oba, and Masafumi Oyamada. 2026. Best-of-∞: Asymptotic Performance of Test-Time LLM Ensembling. In *International Conference on Learning Representations*. OpenReview.net. arXiv:2509.21091.

[12] Jonathan Light, Wei Cheng, Benjamin Riviere, Wu Yue, Masafumi Oyamada, Mengdi Wang, Yisong Yue, Santiago Paternain, and Haifeng Chen. 2025. DISC: Dynamic Decomposition Improves LLM Inference Scaling. In *Advances in Neural Information Processing Systems*. Curran Associates. arXiv:2502.16706.

[13] NEC. 2025. NEC Develops Agent Technology "cotomi Act" for Automating Web-Based Knowledge Work by Learning and Leveraging Tacit Knowledge. https://jpn.nec.com/press/202508/20250827_02.html.

[14] OpenAI. 2025. Computer-Using Agent. https://openai.com/index/computer-using-agent/.

[15] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560* (2023), 16 pages.

[16] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *arXiv preprint arXiv:2303.11366* (2023), 10 pages.

[17] Segev Shlomov, Alon Oved, Sami Marreed, Ido Levy, Offer Akrabi, Avi Yaeli, Łukasz Strąk, Elizabeth Koumpan, Yinon Goldshtein, Eilam Shapira, Nir Mashkif, and Asaf Adi. 2026. From Benchmarks to Business Impact: Deploying IBM Generalist Agent in Enterprise Production. In *AAAI Conference on Artificial Intelligence*. AAAI Press, 40423–40431. https://doi.org/10.1609/aaai.v40i47.41485

[18] Paloma Sodhi, S.R.K. Branavan, Yoav Artzi, and Ryan McDonald. 2024. SteP: Stacked LLM Policies for Web Actions. In *Proceedings of the Conference on Language Modeling (COLM)*. arXiv:2310.03720

[19] Susan Leigh Star and James R. Griesemer. 1989. Institutional Ecology, "Translations," and Boundary Objects: Amateurs and Professionals in Berkeley's Museum of Vertebrate Zoology, 1907–39. *Social Studies of Science* 19, 3 (1989), 387–420.

[20] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291* (2023), 22 pages.

[21] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2024. Agent Workflow Memory. *arXiv preprint arXiv:2409.07429* (2024), 16 pages.

[22] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems*. Curran Associates, Vancouver, Canada, 37 pages.

[23] Ran Xu, Kaixin Ma, Wenhao Yu, Hongming Zhang, Joyce C. Ho, Carl Yang, and Dong Yu. 2025. Retrieval-augmented GUI Agents with Generative Guidelines. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 17866–17875.

[24] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An Illusion of Progress? Assessing the Current State of Web Agents. In *Proceedings of the Conference on Language Modeling (COLM)*. 15 pages.

[25] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. 2025. AgentOccam: A Simple Yet Strong Baseline for LLM-Based Web Agents. In *International Conference on Learning Representations*. OpenReview.net, Singapore, 21 pages.

[26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[27] Haochen Zhang, Masafumi Enomoto, Ryoma Obara, Kunihiro Takeoka, and Masafumi Oyamada. 2025. *WebArena Mod*. https://github.com/nec-research-labs/WebArena-Mod

[28] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM Agents Are Experiential Learners. In *AAAI Conference on Artificial Intelligence*. AAAI Press, Washington, DC, USA, 19632–19642.

[29] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a Generalist Web Agent, if Grounded. *arXiv preprint arXiv:2401.01614* (2024), 18 pages.

[30] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*. OpenReview.net, Vienna, Austria, 26 pages.

A Scaffold Details

This appendix provides implementation details for key scaffold mechanisms introduced in Section 2.

A.1 Test-Time Scaling

Two mechanisms improve per-task accuracy at the cost of additional inference-time computation.

Best-of-N action selection. At each step of the ReAct loop, the scaffold samples N candidate actions from the reasoning model and selects the majority-voted action before execution. This pre-execution consensus reduces the variance of individual samples and filters out hallucinated or imprecise actions (e.g., clicking the wrong element) without requiring an explicit post-action verifier. In practice we use moderate N (typically ≤ 5) to balance accuracy against latency; adaptive schemes inspired by test-time ensembling [11] can further improve this trade-off.

Task decomposition. The scaffold delegates decomposition to the reasoning model itself: when the model emits a structured plan indicating multiple independent sub-goals within a single user instruction, the scaffold spawns parallel sub-agents, each maintaining a focused context window for its sub-goal [12]. If the model identifies only a single goal, execution proceeds without decomposition.

A.2 Workspace Retrieval

In the behavioral-knowledge evaluation (Section 3.2), all three knowledge formats—trajectory, script, and insight—are stored as wiki pages in the workspace; the agent retrieves relevant pages via the same `search_workspace` mechanism described in Section 2 rather than receiving them in its initial context (see Appendix B for coverage details).

B Behavioral-Knowledge Evaluation Protocol

This appendix supplements the controlled study in Section 3.2 with full protocol details.

B.1 Benchmark and Task Categories

We use a six-category subset of WorkArena-L1 [4] over a ServiceNow instance: *dashboard*, *form*, *knowledge*, *list-filter*, *list-sort*, and *service catalog*. This split separates list tasks into filtering and sorting categories and excludes navigation/menu tasks, which primarily test UI navigation rather than reusable procedural knowledge. Each category contains multiple task templates (e.g., *order-sales-laptop*, *order-ipad-mini* within service catalog); evaluation uses 20 random seeds per template.

B.2 Source Traces

The "user behavior" used as input to the knowledge pipeline consists of successful agent trajectories collected by running a browser agent on WorkArena-L1 tasks. We treat these trajectories as a proxy for observed user behavior: each trace records the sequence of pages visited, actions taken, and resulting state changes—the same signal the behavior logger would capture from a real user's browsing session. Real user traces would additionally contain failures, interruptions, and exploratory detours; the agentic ETL pipeline (Section 2) mitigates such noise by weighting behavioral frequency, so that outlier or erratic actions are unlikely to surface as knowledge artifacts. Any artifacts that do slip through are subject to the bidirectional curation loop: users inspect wiki pages and task items in the shared workspace and can correct or remove inaccurate entries before the agent acts on them.

B.3 Domain Coverage

Domain coverage is defined at the category level: $k/6$ coverage means that behavioral knowledge derived from k of the six categories has been written to the shared knowledge workspace as wiki pages. At each coverage step, one category's worth of knowledge artifacts (approximately 34–50 per category, 277 total at 100% coverage) is added in its entirety; there is no sub-sampling within categories. During evaluation, the agent does not receive these artifacts in its initial prompt; instead, it retrieves relevant wiki pages on demand via the `search_workspace` tool (Appendix A), matching the deployed system's retrieval behavior.

B.4 Ordering Control

To control for the effect of which categories are added first, we evaluate six random orderings of the six categories (`order_00`–`order_05`). Each data point in Figure 4 is the mean across these six orderings.

C End-to-End Episode: Service Catalog

Section 3.2 summarizes the service-catalog episode inline. Table 2 provides the per-ordering breakdown. In five of six orderings the agent achieves perfect accuracy immediately upon receiving the scripts, regardless of when in the coverage sequence the service-catalog domain is introduced.

For example, the procedural script for *order-sales-laptop* reads:

(1) Navigate to "Service Catalog."
(2) Select the "Hardware" category.
(3) Select the specific item (e.g., "Sales Laptop").
(4) Set the quantity.
(5) For each software option, check the box if required.
(6) Enter additional software requirements.
(7) Click "Order Now."
(8) On the confirmation page, copy the request number.

Table 2: Service-catalog success rate before and after adding catalog-ordering scripts (script format). Each row is one of six random category orderings.

	Before	After	Final (100%)
order_00	65%	100%	100%
order_01	75%	100%	95%
order_02	70%	100%	100%
order_03	80%	100%	100%
order_04	90%	100%	95%
order_05	70%	95%	95%
Mean	**75.0%**	**99.2%**	**97.5%**